PROJECT DOCUMENTATION

Gatekeeper

A multi-tenant API gateway platform: control plane, admin console and data-plane scaffolds. How it is built, how to run it, and how a change flows through it.

Generated 2026-10-30 from the Markdown files in the docs folder.
Reflects the state of the source code on 2026-10-29.

Contents

Gatekeeper Documentation
- Start here
- Status at a glance
- Regenerating the PDF

1. System Overview
- 1.1 What Gatekeeper is
- 1.2 Components
- 1.3 How the pieces connect
- 1.4 The three roles
- 1.5 Core concepts
- 1.6 What a tenant sees versus what the gateway sees

2. Getting Started
- 2.1 Prerequisites
- 2.2 Database
- 2.3 Running the services
- 2.4 Running the UI
- 2.5 First-run walkthrough
- 2.6 Rebuilding the documentation

3. Backend API Reference
- 3.1 Conventions
- 3.2 Health
- 3.3 Authentication
- 3.4 Platform: organizations (role PLATFORM_ADMIN)
- 3.5 Platform: tenant administrator (role PLATFORM_ADMIN)
- 3.6 Tenant: own organization (role TENANT_ADMIN)
- 3.7 Tenant: routes (role TENANT_ADMIN)
- 3.8 Gateway (role GATEWAY_SERVICE)
- 3.9 Other services
- 3.10 Quick examples

4. Data Model
- 4.1 Entity relationship overview
- 4.2 Tables
- 4.3 Seed data
- 4.4 Versioning rules
- 4.5 Tenant isolation in queries

5. Security and Authentication
- 5.1 Sign-in flow
- 5.2 The token
- 5.3 Authorization
- 5.4 Tenant isolation
- 5.5 Password handling
- 5.6 How the UI handles the session
- 5.7 Security checklist for deployment

6. Frontend (gatekeeper-ui)
- 6.1 Stack
- 6.2 Screens and routes
- 6.3 Folder structure
- 6.4 How data flows through the UI
- 6.5 The route form
- 6.6 Layout and accessibility
- 6.7 Adding a new screen

7. End-to-End Flows

7.1 Onboarding a new tenant (platform admin)
7.2 Creating and publishing a route (tenant admin)
7.3 Changing a route and reading its history
7.4 Deactivating and deleting a route
7.5 How the gateway gets its configuration
7.6 Suspending a tenant
7.7 Deleting a tenant
7.8 Session lifecycle in the UI

8. Known Issues and Roadmap

8.1 Known issues
8.2 Roadmap suggestions
8.3 Documentation maintenance

Gatekeeper Documentation

Gatekeeper is a multi-tenant API gateway platform made of three Spring Boot services and a React admin UI. This folder explains what exists **today**, how the pieces fit together, and how a request or a configuration change flows through the system.

> Last verified against the source code on 2026-10-29. Secrets (database password, JWT signing secret, seeded password hashes) are intentionally not reproduced anywhere in this documentation.

Start here

If you want to...	Read
See the whole picture in one visual page	index.html (open it in a browser)
Read everything as a single file	Gatekeeper-Documentation.pdf
Understand what each service does	01 - System overview
Run the project locally	02 - Getting started
Call the backend	03 - Backend API reference
Understand tables, entities and versioning	04 - Data model
Understand login, JWT, roles and tenant isolation	05 - Security and authentication
Work on the React UI	06 - Frontend
Follow a scenario step by step	07 - End-to-end flows
Know what is missing or risky	08 - Known issues and roadmap

`architecture.md` in this folder is an earlier machine-generated snapshot. It predates the route-management work and the finished UI, so where it disagrees with the files above, the files above are correct.

Status at a glance

Area	Status
Login and JWT issuing	Implemented
Platform admin: organizations (tenants) and tenant administrators	Implemented
Tenant admin: own organization, routes, header rules, version history, audit log	Implemented
Gateway-facing endpoint listing active routes	Implemented
Admin UI for the platform, tenant and gateway roles	Implemented (users and gateway registration pages are placeholders)
smart-gateway-service (the actual traffic proxy)	Scaffold: `/health` only
analytics-service	Scaffold: `/health` only
Kafka, Redis, MongoDB usage	Dependencies only, no code
Tenant user management, gateway registration	Not available in the backend

Regenerating the PDF

The PDF is built from the Markdown files in this folder.

```
cd docs/tools
npm install
npm run build:pdf
```

This needs Google Chrome or Microsoft Edge installed. See 02 - Getting started, section 2.6.

1. System Overview

1.1 What Gatekeeper is

Gatekeeper is designed as a multi-tenant API gateway. The design has two halves:

- **Control plane** (`api-management-service`): where administrators define tenants (organizations), their routes and header rules. It stores configuration, versions every change, and keeps an audit trail.
- **Data plane** (`smart-gateway-service`): the process that would receive client traffic and forward it to the right upstream service according to the routes the control plane publishes.

A third service, `analytics-service` , is meant to collect usage data.

Only the control plane and the admin UI contain real logic today. The gateway and analytics services start and answer `/health` , nothing more.

1.2 Components

Component	Purpose	Port	Datastore	State
`api-management-service`	Control plane: login, tenants, routes, versions, audit	8081	MySQL `gatekeeper_manager`	Implemented
`smart-gateway-service`	Data plane: route traffic, rate limit, idempotency	8080	Redis (dependency only)	Scaffold
`analytics-service`	Usage analytics	8082	MongoDB (dependency only, auto-config excluded)	Scaffold
`gatekeeper-ui`	Admin console for all three roles	3000 (dev)	none (token in `sessionStorage`)	Implemented

Technology: Java 21, Spring Boot 4.1.2 (Spring MVC, Spring Data JPA, Flyway, Spring Security with OAuth2 resource server, Bean Validation, Lombok) for the services; React 19, TypeScript, Vite 8, Tailwind CSS 4 and react-router-dom 7 for the UI.

Java package root: `in.org.gatekeeper.<service_name_with_underscores>` , for example `in.org.gatekeeper.api_management_service` .

1.3 How the pieces connect



Everything drawn with a real connection (UI to control plane, control plane to MySQL) works today. The gateway-to-control-plane link and the gateway-to-upstream link are the intended design: the endpoint the gateway should call exists, but the gateway does not call it.

1.4 The three roles

Every account has exactly one role. The role decides which part of the API and which part of the UI the account can use.

Role	Who	Can use	UI area
`PLATFORM_ADMIN`	Operator of the whole platform	`/api/platform/**` : create and manage organizations and their tenant administrator	`/platform/...`
`TENANT_ADMIN`	One administrator per organization	`/api/tenant/**` : own organization, routes, version history, audit log	`/tenant/...`
`GATEWAY_SERVICE`	The data-plane service account	`/api/gateway/**` : read the list of active routes	`/gateway/...`

Two accounts are created by database migration: `platform-admin` (PLATFORM_ADMIN) and `gateway-service` (GATEWAY_SERVICE). Tenant administrators are created by a platform admin through the API or the UI.

1.5 Core concepts

Organization (tenant). A customer of the platform. Has a unique `name` and a unique `slug` (lowercase letters, digits, single hyphens). Status is one of `PENDING` , `ACTIVE` , `SUSPENDED` , `DELETED` . New organizations start `PENDING` . Deleting is a soft delete: the row stays with status `DELETED` .

Tenant administrator. The single login account of an organization (the database enforces one per organization). Can only sign in while the organization is `ACTIVE` .

Route. A rule telling the gateway how to forward a request:

Field	Meaning
`routeId`	Business identifier, unique inside an organization, cannot be changed after creation
`name`	Display name

Field	Meaning
`pathPattern`	Incoming path to match. Must start with `/<organization-slug>/`
`targetBaseUrl`	Upstream base URL, `http` or `https`
`targetPathPrefix`	Path prefix on the upstream, must start with `/`
`allowedMethods`	Set of HTTP methods the route accepts
`requestsPerMinute`	Optional rate limit (null means no limit)
`responseTimeoutMs`	Upstream timeout in milliseconds
`idempotencyEnabled`	Whether idempotency handling is on
`active`	Only active routes of active organizations are published to the gateway
`headerRules`	List of rules that add or remove headers on the request or the response
`configurationVersion`	Counter that increases on every change

Configuration version. Every create, update, activate, deactivate and delete writes a snapshot (the full route as JSON) to `route_configuration_versions`. History is keyed by the business `routeId`, so it survives deleting the route.

Audit record. Every mutating action by the control plane writes who did it, what, on which resource, how long it took, the result and an optional `X-Correlation-ID` header value.

1.6 What a tenant sees versus what the gateway sees

- A tenant administrator sees **only their own** organization, routes, versions and audit records. The organization is taken from the `organizationId` claim in the JWT, never from the URL or the request body.
- The gateway account sees **all active routes of all** `ACTIVE` **organizations** in a single list, each tagged with its `organizationId`.
- Deactivating a route, suspending an organization or deleting an organization removes the affected routes from what the gateway receives on its next fetch.

2. Getting Started

2.1 Prerequisites

Tool	Version	Used for
JDK	21	The three Spring Boot services
MySQL	8.x	`api-management-service` (the only service that needs a database)
Node.js and npm	a current LTS	`gatekeeper-ui`
Chrome or Edge	any recent	Only for rebuilding the PDF

Maven is not required: each service ships a Maven wrapper (`mvnw`, `mvnw.cmd`).

2.2 Database

`api-management-service` connects to `jdbc:mysql://localhost:3306/gatekeeper_manager` as configured in `api-management-service/src/main/resources/application.properties`.

1. Create an empty database named `gatekeeper_manager`. The JDBC URL does not create it for you.
2. Copy `api-management-service/.env.example` to `.env` and set `DB_URL` (a `jdbc:mysql://...` URL), `DB_USERNAME`, `DB_PASSWORD` and `JWT_SECRET` (at least 32 random characters). `.env` is git-ignored and is read from the folder you start the service in. Real environment variables work too and take precedence.
3. Start the service from the `api-management-service` folder. Flyway runs automatically and applies:
   - `V1__create_api_management_schema.sql`: all tables
   - `V2__seed_management_roles.sql`: the three roles
   - `V3__seed_platform_accounts.sql`: the `platform-admin` and `gateway-service` accounts

Hibernate is set to `ddl-auto=validate`, so Flyway owns the schema and the application refuses to start if an entity does not match a table.

The seeded accounts' passwords are not documented here. They are stored only as BCrypt hashes in `V3__seed_platform_accounts.sql`; ask the team that created the migration, or insert your own hash.

2.3 Running the services

From each service folder:

```
cd api-management-service
./mvnw spring-boot:run          # Windows: mvnw.cmd spring-boot:run
```

Service	Port	Check
api-management-service	8081	`GET http://localhost:8081/health` returns service and database status
smart-gateway-service	8080	`GET http://localhost:8080/health`
analytics-service	8082	`GET http://localhost:8082/health`

Only `api-management-service` is needed to use the UI. The other two start without any external dependency.

2.4 Running the UI

```
cd gatekeeper-ui
npm install
npm run dev
```

Open http://localhost:3000. The dev server proxies every `/api` request to `http://localhost:8081`, so no CORS setup is needed. The port is strict: if 3000 is taken, Vite refuses to start rather than moving to another port.

Script	What it does
`npm run dev`	Dev server with hot reload
`npm run build`	`tsc -b` type check, then production bundle in `dist/`
`npm run lint`	oxlint
`npm run preview`	Serve the production bundle locally

To point the UI at a backend that is not on `localhost:8081` (for example a production build served from a different host), set `VITE_API_BASE_URL` in `gatekeeper-ui/.env` (no trailing slash). Leave it empty in development to use the proxy. A production deployment must then either serve the UI and API from the same origin or add CORS to the backend, which does not have any CORS configuration today.

2.5 First-run walkthrough

1. Sign in as `platform-admin`. You land on **Organizations**.
2. **Create organization**: give it a name; the slug is suggested from the name. The organization is `PENDING`.
3. Open the organization. In the **Tenant admin** card, fill in the form (username, password of 8 to 100 characters).
4. Back on the organization, **Change status** to `ACTIVE`. A tenant administrator can only sign in while the organization is active.
5. Sign out, sign in as the new tenant administrator. You land on the **Tenant dashboard**.
6. Open **Routes**, **Create route**. The path pattern must begin with `/<your-slug>/`. New routes are inactive; **Activate** the route.
7. Sign out, sign in as `gateway-service`. **Active routes** now lists the route you activated.

2.6 Rebuilding the documentation

The PDF and the combined HTML are generated from the Markdown files in this folder.

```
cd docs/tools
npm install
npm run build:pdf
```

The script renders every `*.md` file in the documented order into one HTML document, then prints it to `docs/Gatekeeper-Documentation.pdf` with headless Chrome or Edge. Set the `CHROME_PATH` environment variable if neither is found in the default Windows install locations.

`docs/index.html` is hand-written and is not touched by the script. Edit it directly.

3. Backend API Reference

All endpoints belong to `api-management-service` (base URL `http://localhost:8081`). The other two services expose only `GET /health`.

3.1 Conventions

- **Format:** JSON request and response bodies.
- **Authentication:** `Authorization: Bearer <token>` from `POST /api/auth/login`. See 05 - Security and authentication.
- **Authorization** is decided by URL prefix:

URL prefix	Required role
`/health`, `/api/auth/login`	none (public)
`/api/platform/**`	`PLATFORM_ADMIN`
`/api/tenant/**`	`TENANT_ADMIN`
`/api/gateway/**`	`GATEWAY_SERVICE`
anything else	any authenticated user

- **Timestamps** are ISO local date-times without a zone, for example `2026-10-29T14:48:03.123456`.
- **Validation** uses Bean Validation (`@Valid`). A body that fails validation is rejected with HTTP 400.
- **Error statuses.** The code throws `ApiException(status, message)` for business errors. The statuses listed below as "Intended" are what the code asks for. Because no `@ControllerAdvice` exists yet, see 08 - Known issues, clients may receive a generic 500 instead.

3.2 Health

GET /health (public)

Response `200`:

```
{ "serviceStatus": "UP", "dbStatus": "UP", "timestamp": "2026-10-29T14:48:03" }
```

`dbStatus` is `DOWN` when a `SELECT 1` fails.

3.3 Authentication

POST /api/auth/login (public)

Request:

```
{ "username": "string (required)", "password": "string (required)" }
```

Response `200`:

```
{
  "token": "<jwt>",
  "tokenType": "Bearer",
  "expiresInSeconds": 3600,
  "role": "TENANT_ADMIN",
  "organizationId": 7
}
```

`organizationId` is `null` for platform and gateway accounts.

Intended errors: `401` unknown user, disabled user or wrong password (same message for all three: "Invalid username or password"); `403` the user's organization is not `ACTIVE`.

3.4 Platform: organizations (role PLATFORM_ADMIN)

`OrganizationResponse`:

```
{ "id": 7, "name": "Acme Corp", "slug": "acme-corp", "status": "PENDING",
  "createdAt": "...", "updatedAt": "..." }
```

Method	Path	Body	Success	Notes
POST	`/api/platform/organizations`	`OrganizationRequest`	201 `OrganizationResponse`	Created as `PENDING`. Intended 409 if name or slug already exists (case-insensitive)
GET	`/api/platform/organizations`	-	200 list	Excludes `DELETED`. Not paginated
GET	`/api/platform/organizations/{id}`	-	200	Intended 404 if missing or `DELETED`
PUT	`/api/platform/organizations/{id}`	`OrganizationRequest`	200	Uniqueness check ignores the organization itself
PATCH	`/api/platform/organizations/{id}/status`	`OrganizationStatusRequest`	200	Any of `PENDING`, `ACTIVE`, `SUSPENDED`; `DELETED` is rejected (intended 400)
DELETE	`/api/platform/organizations/{id}`	-	204	Soft delete

`OrganizationRequest`: `name` (required, at most 150 characters), `slug` (required, at most 100, pattern `^[a-z0-9]+(?:-[a-z0-9]+)*$`). `OrganizationStatusRequest`: `{ "status": "ACTIVE" }`, required.

3.5 Platform: tenant administrator (role PLATFORM_ADMIN)

An organization has at most one tenant administrator.

`TenantAdminRequest`: `username` (required, at most 100), `password` (required, 8 to 100 characters), `enabled` (default `true`). `TenantAdminResponse`: `{ "id": 3, "organizationId": 7, "username": "acme-admin", "enabled": true }`. The password hash is never returned.

Method	Path	Success	Notes
POST	`/api/platform/organizations/{organizationId}/tenant-admin`	201	Intended 409 if the organization already has an admin or the username is taken
GET	`/api/platform/organizations/{organizationId}/tenant-admin`	200	Intended 404 if none
PUT	`/api/platform/organizations/{organizationId}/tenant-admin`	200	Password is required on every update and is re-hashed
DELETE	`/api/platform/organizations/{organizationId}/tenant-admin`	204	Hard delete of the user

3.6 Tenant: own organization (role TENANT_ADMIN)

The organization is resolved from the token, not from the URL.

Method	Path	Body	Success	Notes
GET	`/api/tenant/organization`	-	200 `OrganizationResponse`	
PUT	`/api/tenant/organization`	`TenantOrganizationRequest`	200	Only the **name** can be changed. Intended 409 if another organization already has that name

`TenantOrganizationRequest`: `{ "name": "required, at most 150 characters" }`.

3.7 Tenant: routes (role TENANT_ADMIN)

All operations are limited to the caller's organization. A route that belongs to another organization is reported as not found.

Request body: `RouteRequest`

Field	Type	Rules
`routeId`	string	required, at most 100. On update it must equal the stored value
`name`	string	required, at most 200
`pathPattern`	string	required, at most 500, must start with `/<organization-slug>/`
`targetBaseUrl`	string	required, at most 1000, must be a valid `http` or `https` URL
`targetPathPrefix`	string	required, at most 500, must start with `/`
`requestsPerMinute`	integer or null	at least 1 when present
`responseTimeoutMs`	integer	required, at least 1
`idempotencyEnabled`	boolean	default `false`

Field	Type	Rules
`active`	boolean	default `false`
`allowedMethods`	array of strings	required, not empty; upper-cased on save
`headerRules`	array of `HeaderRuleRequest`	required (may be empty)

`HeaderRuleRequest`:

Field	Rules
`direction`	required, `REQUEST` or `RESPONSE` (case-insensitive, stored upper-case)
`action`	required, `ADD` or `REMOVE` (case-insensitive, stored upper-case)
`headerName`	required, at most 200
`headerValue`	at most 2000. **Required when action is** `ADD`

Response body: `RouteResponse`

```
{
  "id": 12, "organizationId": 7, "routeId": "orders-v1", "name": "Orders API",
  "pathPattern": "/acme-corp/orders", "targetBaseUrl": "https://orders.internal",
  "targetPathPrefix": "/v1", "requestsPerMinute": 600, "responseTimeoutMs": 5000,
  "idempotencyEnabled": true, "active": true, "configurationVersion": 3,
  "allowedMethods": ["GET", "POST"],
  "headerRules": [ { "id": 1, "direction": "REQUEST", "action": "ADD",
                     "headerName": "X-Tenant", "headerValue": "acme" } ],
  "createdAt": "...", "updatedAt": "..."
}
```

Endpoints

Method	Path	Body	Success	What it does
POST	`/api/tenant/routes`	`RouteRequest`	201 `RouteResponse`	Creates the route at version 1. Intended 409 if the `routeId` already exists in the organization
GET	`/api/tenant/routes`	-	200 list	All routes of the organization, any status
GET	`/api/tenant/routes/{id}`	-	200	`{id}` is the numeric database id
PUT	`/api/tenant/routes/{id}`	`RouteRequest`	200	Replaces the whole route including header rules; version + 1. Intended 400 if `routeId` differs
PATCH	`/api/tenant/routes/{id}/status`	`{ "active": true }`	200	Activates or deactivates; version + 1
DELETE	`/api/tenant/routes/{id}`	-	204	Records a final `DELETED` version, then removes the route
GET	`/api/tenant/route-versions/{routeId}`	-	200 list	`{routeId}` is the **business** id (for example `orders-v1`), newest first. Still works after the route was deleted
GET	`/api/tenant/audit-records`	-	200 list	All audit records of the organization, newest first

`RouteVersionResponse`:

```
{ "id": 40, "routeId": "orders-v1", "versionNumber": 3, "changeType": "UPDATED",
  "configurationSnapshot": "{\"id\":12,...}", "createdAt": "..." }
```

`changeType` is one of `CREATED`, `UPDATED`, `ACTIVATED`, `DEACTIVATED`, `DELETED`. `configurationSnapshot` is a **string** containing the route as JSON; parse it to read it.

`AuditRecordResponse`:

```
{ "id": 90, "actor": "acme-admin", "action": "UPDATE_ROUTE", "resourceType": "API_ROUTE",
  "resourceId": "orders-v1", "correlationId": null, "durationMs": 14, "result": "SUCCESS",
  "details": "Route configuration version 3 created", "createdAt": "..." }
```

Audit actions written today:

Action	Written by
`CREATE_ROUTE`, `UPDATE_ROUTE`, `ACTIVATE_ROUTE`, `DEACTIVATE_ROUTE`, `DELETE_ROUTE`	Tenant route operations
`UPDATE_OWN_ORGANIZATION`	Tenant renames its organization
`CREATE_ORGANIZATION`, `UPDATE_ORGANIZATION`, `CHANGE_ORGANIZATION_STATUS`, `DELETE_ORGANIZATION`	Platform organization operations
`CREATE_TENANT_ADMIN`, `UPDATE_TENANT_ADMIN`, `DELETE_TENANT_ADMIN`	Platform tenant-admin operations

Platform actions are recorded against the affected organization, so a tenant's audit log also shows what platform administrators did to it. `result` is always `SUCCESS` today; failed operations are not recorded. If the request carries an `X-Correlation-ID` header, its value is stored in `correlationId`.

3.8 Gateway (role GATEWAY_SERVICE)

GET /api/gateway/routes

Returns a list of `RouteResponse` for every route where `active = true` **and** the owning organization is `ACTIVE`. Each item carries its `organizationId`. This is the configuration feed the data plane is meant to poll.

3.9 Other services

Service	Endpoint	Response
smart-gateway-service (8080)	`GET /health`	`serviceName`, `serviceDescription`, `appStatus: "UP"`
analytics-service (8082)	`GET /health`	`appName`, `appDescription`, `appHealth: "UP"`

The field names differ per service. Everything else on these two services requires authentication, and no token validation is configured yet, so every non-health request is rejected.

3.10 Quick examples

```
# 1. sign in
curl -s -X POST http://localhost:8081/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"acme-admin","password":"<password>"}'

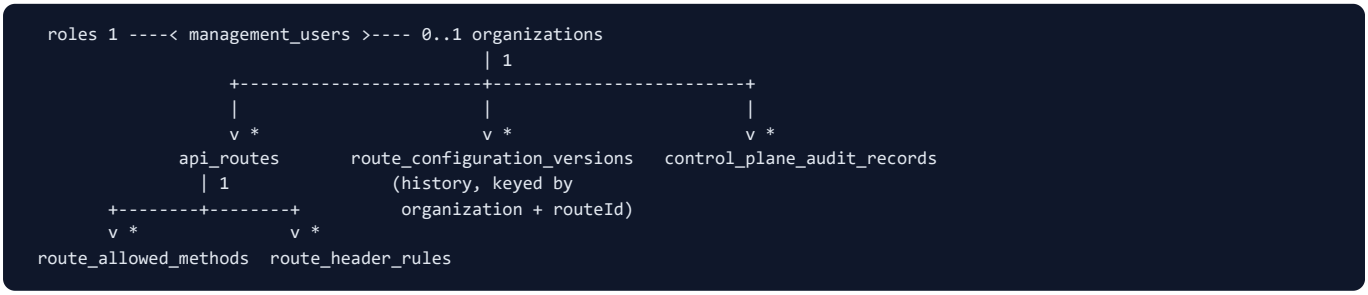
# 2. create a route (use the token from step 1)
curl -s -X POST http://localhost:8081/api/tenant/routes \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"routeId":"orders-v1","name":"Orders API","pathPattern":"/acme-corp/orders",
       "targetBaseUrl":"https://orders.internal","targetPathPrefix":"/v1",
       "requestsPerMinute":600,"responseTimeoutMs":5000,"idempotencyEnabled":false,
       "active":false,"allowedMethods":["GET","POST"],
       "headerRules":[{"direction":"REQUEST","action":"ADD","headerName":"X-Tenant","headerValue":"acme"}]}'

# 3. activate it
curl -s -X PATCH http://localhost:8081/api/tenant/routes/12/status \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"active":true}'
```

4. Data Model

Only `api-management-service` has a database (MySQL, schema `gatekeeper_manager`). Flyway owns the schema; Hibernate only validates it (`ddl-auto=validate`). Every table is mapped by a JPA entity.

4.1 Entity relationship overview



`route_configuration_versions` deliberately has **no foreign key to** `api_routes`: it refers to the route by its business `route_id`, so history stays readable after the route is deleted.

4.2 Tables

organizations (entity `Organization`)

Column	Notes
`id`	primary key, auto-increment
`name`	unique
`slug`	unique; becomes the mandatory first segment of every route `path_pattern`
`status`	`PENDING`, `ACTIVE`, `SUSPENDED`, `DELETED` (stored as text)
`created_at`, `updated_at`, `deleted_at`	`deleted_at` is set on soft delete

roles (entity `Role`)

`id`, `name` (unique). Seeded: `PLATFORM_ADMIN`, `TENANT_ADMIN`, `GATEWAY_SERVICE`.

management_users (entity `ManagementUser`)

Column	Notes
`id`, `username` (unique)	
`password_hash`	BCrypt
`enabled`	disabled users cannot sign in
`organization_id`	nullable and **unique**: at most one user per organization; `NULL` for platform and gateway accounts
`role_id`	required
`created_at`, `updated_at`	

api_routes (entity `ApiRoute`)

Column	Notes
`id`	primary key
`organization_id`	owner; `(organization_id, route_id)` is unique
`route_id`, `name`, `path_pattern`, `target_base_url`, `target_path_prefix`	see 01 - Core concepts
`requests_per_minute`	nullable
`response_timeout_ms`	required
`idempotency_enabled`, `active`	both default false
`configuration_version`	starts at 1, +1 on every change
`created_at`, `updated_at`	

route_allowed_methods (element collection of `ApiRoute`)

Primary key `(route_pk, http_method)`. Deleted with the route (`ON DELETE CASCADE`). Loaded eagerly.

route_header_rules (entity `RouteHeaderRule`)

`id`, `route_pk` (cascade delete), `direction` (`REQUEST` / `RESPONSE`), `action` (`ADD` / `REMOVE`), `header_name`, `header_value` (nullable), timestamps. On every route update the rule list is cleared and rebuilt from the request (`orphanRemoval`), so rule ids change on each save.

route_configuration_versions (entity `RouteConfigurationVersion`)

Column	Notes
`organization_id`, `route_id`	which route
`version_number`	unique per `(organization_id, route_id)`
`change_type`	`CREATED`, `UPDATED`, `ACTIVATED`, `DEACTIVATED`, `DELETED`
`configuration_snapshot`	`LONGTEXT`: the complete `RouteResponse` as JSON at that moment
`created_at`	

control_plane_audit_records (entity `ControlPlaneAuditRecord`)

`organization_id` (nullable), `actor` (username), `action`, `resource_type`, `resource_id`, `correlation_id` (from `X-Correlation-ID`), `duration_ms`, `result`, `details`, `created_at`.

4.3 Seed data

Migration	Content
`V1__create_api_management_schema.sql`	All tables above
`V2__seed_management_roles.sql`	The three roles
`V3__seed_platform_accounts.sql`	`platform-admin` and `gateway-service`, both with `organization_id = NULL`

4.4 Versioning rules

Action	`configuration_version`	Version row written
Create route	set to 1	`CREATED`
Update route (PUT)	+1	`UPDATED`
Activate (PATCH status, true)	+1	`ACTIVATED`
Deactivate (PATCH status, false)	+1	`DEACTIVATED`
Delete route	+1	`DELETED` (final snapshot), then the route row is removed

Version numbers therefore never repeat for a route, and the highest number always matches `configuration_version` on the live route (or the final `DELETED` version for a removed route). Re-creating a route with the same `routeId` after deleting it hits the unique `(organization, route, version)` constraint, because the new route starts again at version 1; see 08 - Known issues.

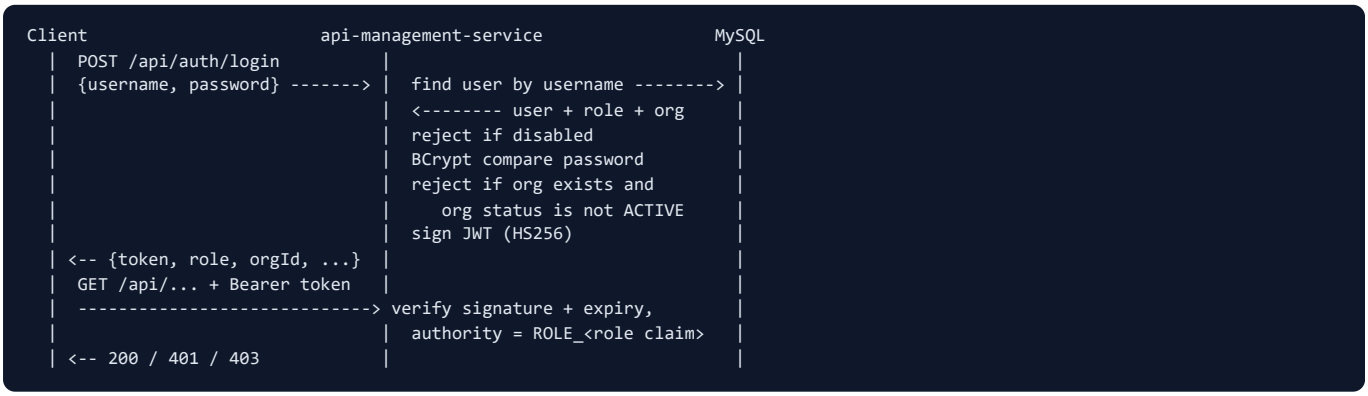
4.5 Tenant isolation in queries

Route, version and audit queries always include the organization id taken from the JWT (`findByIdAndOrganizationId` , `findAllByOrganizationId` , `findAllByOrganizationIdAndRouteIdOrderByVersionNumberDesc` , `findAllByOrganizationIdOrderByCreatedAtDesc`). The only cross-tenant query is `findAllByActiveTrueAndOrganizationStatus(ACTIVE)` , used by the gateway endpoint.

5. Security and Authentication

Authentication and authorization are fully implemented in `api-management-service` only. The gateway and analytics services have a security configuration but no token validation (see 08 - Known issues).

5.1 Sign-in flow



Rules enforced at login:

1. Unknown user, disabled user and wrong password all produce the same "Invalid username or password" (intended 401), so the response does not reveal which accounts exist.
2. A user that belongs to an organization can sign in only while that organization is `ACTIVE`; otherwise "The organization is not active" (intended 403).
3. Platform and gateway accounts have no organization and skip rule 2.

5.2 The token

Property	Value
Algorithm	HS256 (HMAC with a shared secret from `security.jwt.secret`)
Lifetime	`security.jwt.expiration-seconds`, currently 3600 (one hour)
`iss`	`gatekeeper-api-management`
`sub`	the username
`iat`, `exp`	issue and expiry time
`role`	`PLATFORM_ADMIN`, `TENANT_ADMIN` or `GATEWAY_SERVICE`
`organizationId`	present only for users that belong to an organization

There is no refresh token, no logout endpoint and no revocation list. A token stays valid until it expires, even if the user is later disabled or the organization is suspended. (Suspending an organization does, however, block tenant data access immediately, see 5.4.)

5.3 Authorization

`SecurityConfig` maps URL prefixes to roles (table in 03 - Backend API reference). The `role` claim becomes the authority `ROLE_<role>`, so `hasRole("TENANT_ADMIN")` matches a token whose claim is `TENANT_ADMIN`. There is no method-level security.

The service is stateless (no HTTP session) and CSRF protection is disabled, which is appropriate for a bearer-token API that does not use cookies.

5.4 Tenant isolation

Every tenant endpoint calls `TenantContext.getOrganization()`, which:

1. Reads the `organizationId` claim from the authenticated JWT. Missing means the request is refused ("Tenant identity is missing").
2. Loads the organization from the database. Missing means refused ("Organization does not exist").
3. Requires its status to be `ACTIVE`. Otherwise refused ("The organization is not active").

The organization id never comes from the URL, query string or request body, so a tenant cannot address another tenant's data by changing an id. Lookups by id also include the organization id (`findByIdAndOrganizationId`), so a foreign route id behaves as "not found".

Because step 3 runs on every request, suspending or deleting an organization takes effect immediately for tenant endpoints even though the user's token has not expired.

5.5 Password handling

Passwords are hashed with BCrypt before storage. Responses never include hashes. Creating or updating a tenant administrator requires a password of 8 to 100 characters, and an update always requires a new password.

5.6 How the UI handles the session

Concern	Behavior
Storage	The session (token, username, role, organization id, issue and expiry times) is kept in `sessionStorage` under `gatekeeper.session`. It disappears when the browser tab closes
Expiry	The session expires at the earlier of the response's `expiresInSeconds` and the token's `exp` claim. A timer signs the user out then with a "session expired" notice. A stored session that has already expired is discarded on load
401 from the API	Any 401 on an authenticated call clears the session and returns to the login page
Route protection	`ProtectedRoute` allows only the listed roles. A signed-in user who opens another role's area is redirected to their own home. `GuestRoute` keeps signed-in users away from `/login`
Role homes	`PLATFORM_ADMIN` goes to `/platform/organizations`, `TENANT_ADMIN` to `/tenant`, `GATEWAY_SERVICE` to `/gateway`
Token display	The token is never rendered

Client-side route protection is a convenience only. The server enforces every permission.

5.7 Security checklist for deployment

- The database password and JWT secret are read from `DB_PASSWORD` and `JWT_SECRET` (a git-ignored `.env` file locally, real environment variables or a secret store in deployment). Use a fresh, long random `JWT_SECRET` outside local development; the value used during early development should be treated as exposed.
- Change the seeded `platform-admin` and `gateway-service` passwords.
- Serve the UI and API over HTTPS.
- Decide on CORS if the UI will be served from a different origin than the API; none is configured.
- Wire JWT validation into the gateway and analytics services before adding endpoints there. Both would need the shared HS256 secret, or the design should move to asymmetric keys.

6. Frontend (gatekeeper-ui)

6.1 Stack

React 19, TypeScript (strict mode), Vite 8, Tailwind CSS 4 (imported in `src/index.css`), react-router-dom 7 (data router with lazy-loaded pages), oxlint. There is no state-management or data-fetching library: plain hooks and a small `fetch` wrapper.

6.2 Screens and routes

URL	Role	Screen	Backend used
`/login`	guest	Sign in	`POST /api/auth/login`
`/platform/organizations`	PLATFORM_ADMIN	Organization list with search, status filter, sort; create, edit, change status, delete	`/api/platform/organizations`
`/platform/organizations/:id`	PLATFORM_ADMIN	Organization detail plus the **Tenant admin** card (create, edit, delete)	organizations and `.../tenant-admin`
`/tenant`	TENANT_ADMIN	Dashboard: organization summary, route counts, five most recent audit entries, session card	`/api/tenant/organization`, `/routes`, `/audit-records`
`/tenant/routes`	TENANT_ADMIN	Route list: search, status filter, sort; create, edit, activate or deactivate, delete	`/api/tenant/routes`
`/tenant/routes/:id`	TENANT_ADMIN	Route detail: all settings, header rules table, version history with snapshot viewer	`/routes/{id}`, `/route-versions/{routeId}`
`/tenant/audit`	TENANT_ADMIN	Audit log: search by actor, resource or details; filter by action	`/api/tenant/audit-records`
`/tenant/organization`	TENANT_ADMIN	Organization details and rename	`/api/tenant/organization`
`/tenant/users`	TENANT_ADMIN	Placeholder ("Coming soon")	none: no backend endpoint
`/gateway`	GATEWAY_SERVICE	Session information	none
`/gateway/routes`	GATEWAY_SERVICE	Read-only list of active routes across tenants	`/api/gateway/routes`
`/gateway/registration`	GATEWAY_SERVICE	Placeholder ("Coming soon")	none: no backend endpoint
`*`	any	Not found	none

6.3 Folder structure

The components folder is spelled `componenets` in the repository. That spelling is kept; imports must use it.

```
gatekeeper-ui/src
  main.tsx, App.tsx          providers: Toast > Auth > Router
  index.css                  Tailwind import and base styles
  api/
    client.ts                fetch wrapper, ApiError, error-message extraction
    types.ts                 TypeScript mirrors of backend DTOs and enums
    auth.ts                  login
    organizations.ts         platform organization calls
    tenantAdmin.ts           platform tenant-admin calls
    tenantOrganization.ts    tenant: own organization
    routes.ts                tenant: routes, versions, audit records
    gateway.ts               gateway: active routes
  auth/                      AuthProvider, session storage, JWT parsing, role labels and homes
  routes/                    router.tsx (route table, nav menus), ProtectedRoute.tsx
  hooks/                     useAsyncData, useCountdown, useDocumentTitle
  lib/                       format.ts (dates), validation.ts (form rules), dom.ts, guards.ts
  componenets/
    ui/                      Button, Modal, ConfirmDialog, Card, TextField, SelectField,
                             CheckboxField, Banner, EmptyState, StatusBadge, Spinner, Toasts
    layout/AppShell.tsx      sidebar, header, mobile menu, skip link
    organizations/           organization form, status and delete dialogs
    tenant-admin/            tenant admin form and panel
    routes/                  RouteFormModal, RouteStatusDialog, DeleteRouteDialog, RouteVersionsPanel
    session/SessionCard.tsx
  pages/
    LoginPage.tsx, NotFoundPage.tsx
    platform/                OrganizationsPage, OrganizationDetailPage
    tenant/                  Dashboard, Routes, RouteDetail, Audit, Organization, Users
    gateway/                 GatewayStatus, GatewayRoutes, GatewayRegistration
```

6.4 How data flows through the UI

```
Page component
   |  useAsyncData(loader)            loading / error / data / reload()
   v
api/<module>.ts   typed function, e.g. listRoutes(signal)
   v
api/client.ts     adds Authorization header from sessionStorage
   |              aborts on unmount (AbortSignal)
   |              401  -> clear session, back to /login
   |              !ok  -> ApiError(status, friendly message)
   v
fetch  /api/...   (Vite proxy to :8081 in development)
```

Key behaviors:

- `useAsyncData` runs a loader on mount and when the loader function changes, cancels the previous request, and exposes `reload()`. Pages wrap loaders in `useCallback` when they depend on a route parameter.
- **Mutations** are done inside dialogs. Each dialog owns its submitting and error state, shows server errors inside the dialog, and on success calls back to the page, which shows a toast, closes the dialog and reloads the list.
- **Error messages** come from the response body (`message`, `detail`, `error` or the first entry of `errors`) when it is specific; otherwise from a per-call message (for example 409 on route creation) or a status default. Generic server phrases such as "Internal Server Error" are ignored in favour of the friendly text.
- **Forms** validate on submit with the same limits as the backend (`lib/validation.ts`), show field errors, and focus the first invalid field.

6.5 The route form

`RouteFormModal` is used for both create and edit.

Behavior	Detail
Path pattern	Pre-filled with `/<slug>/` when the organization slug is known; the client checks the prefix, the server checks it again

Behavior	Detail
Route ID	Editable on create, read-only on edit (the backend forbids changing it)
Defaults	Methods: `GET`; timeout 5000 ms; rate limit empty (no limit); idempotency off; **active off**
Methods	Checkboxes for GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS; at least one required
Header rules	Add and remove rows. Each row has direction, action, header name and, for `ADD`, a required value. `REMOVE` rows send `headerValue: null`
Saving	Create calls POST; edit calls PUT with the full route; the new version appears in the detail page history

Activating and deactivating is a separate confirmed action (`PATCH .../status`), also available from the list and the detail page.

6.6 Layout and accessibility

- `AppShell` provides a fixed sidebar on wide screens and a toggled menu on small screens, with a "Skip to content" link. The menu items differ per role (`PLATFORM_NAV` , `TENANT_NAV` , `GATEWAY_NAV` in `routes/router.tsx`).
- Modals trap focus, close on Escape and restore focus; they cannot be dismissed while a request is in flight.
- Tables have captions and scoped headers; row actions have visually hidden names ("Edit Orders API").
- Result counts are announced through `aria-live` . Reduced-motion preferences are respected.

6.7 Adding a new screen

1. Add the DTO types to `api/types.ts` and a function to the right `api/*.ts` module.
2. Add field rules to `lib/validation.ts` if there is a form.
3. Build the page in `pages/<area>/` with a default export, using `useAsyncData` , `PageHeader` , `Card` , `EmptyState` , `Banner` and the dialog pattern from an existing page.
4. Register it in `routes/router.tsx` (lazy `page(import(...))`) under the right `ProtectedRoute` , and add a nav entry if it should appear in the sidebar.
5. Run `npm run build` and `npm run lint` .

7. End-to-End Flows

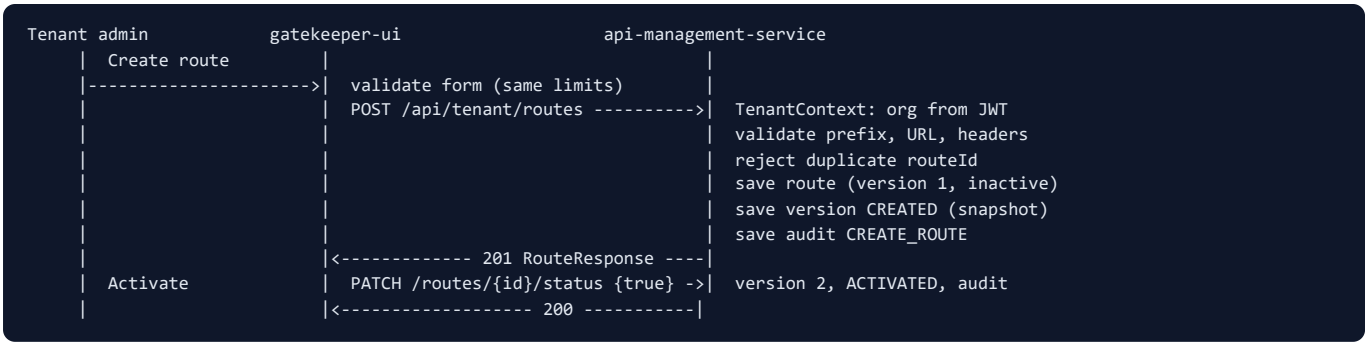
Each flow lists what the user does, which API call the UI makes, and what the backend does. "Intended" outcomes are described in 08 - Known issues.

7.1 Onboarding a new tenant (platform admin)

#	Actor	Action	API call	Backend effect
1	Platform admin	Signs in	`POST /api/auth/login`	JWT with role `PLATFORM_ADMIN`, no organization
2	Platform admin	Create organization (name, slug)	`POST /api/platform/organizations`	Row in `organizations`, status `PENDING`. Audit `CREATE_ORGANIZATION`
3	Platform admin	On the organization page, fills the **Tenant admin** form	`POST /api/platform/organizations/{id}/tenant-admin`	Row in `management_users` (role `TENANT_ADMIN`, BCrypt hash, linked to the organization). Audit `CREATE_TENANT_ADMIN`
4	Platform admin	Change status to `ACTIVE`	`PATCH /api/platform/organizations/{id}/status`	Status updated. Audit `CHANGE_ORGANIZATION_STATUS`
5	Tenant admin	Signs in	`POST /api/auth/login`	Allowed now because the organization is `ACTIVE`. JWT carries `organizationId`

If step 5 happens before step 4, login is refused with "The organization is not active".

7.2 Creating and publishing a route (tenant admin)



Validation performed by the backend on create and update:

1. `pathPattern` starts with `/<organization-slug>/`.
2. `targetPathPrefix` starts with `/`.
3. `targetBaseUrl` parses as a URL with scheme `http` or `https`.
4. Every header rule has direction `REQUEST` / `RESPONSE` and action `ADD` / `REMOVE`; `ADD` rules have a non-blank value.
5. The route id is not already used in the organization (create), or is unchanged (update).

The whole operation (route, versions, audit record) runs in one database transaction.

7.3 Changing a route and reading its history

1. Tenant opens `/tenant/routes/{id}` and chooses **Edit**. The form is pre-filled from `GET /api/tenant/routes/{id}`.

2. Saving sends `PUT /api/tenant/routes/{id}` with the full route. The backend increments the version, replaces the allowed methods and rebuilds the header rules, saves an `UPDATED` snapshot and an `UPDATE_ROUTE` audit record.
3. The detail page reloads. The **Version history** card refreshes because the route's `configurationVersion` changed, and lists versions newest first. **View snapshot** shows the full route as it was at that version, as formatted JSON.

To see what changed between two versions, open both snapshots and compare them. The UI does not compute diffs.

7.4 Deactivating and deleting a route

- **Deactivate:** `PATCH .../status {"active": false}`. Version +1 (`DEACTIVATED`). The route disappears from the gateway feed on its next fetch but remains fully editable.
- **Delete:** `DELETE /api/tenant/routes/{id}`. A final `DELETED` version with the last configuration is saved, an audit record `DELETE_ROUTE` is written, and the route (with its methods and header rules) is removed. The history stays available through `GET /api/tenant/route-versions/{routeId}` and the audit log.

7.5 How the gateway gets its configuration

```
smart-gateway-service (intended)              api-management-service
      |  POST /api/auth/login (gateway-service) ------->|
      |<---------------------------- JWT role GATEWAY_SERVICE
      |  GET /api/gateway/routes + Bearer ------------->|  routes WHERE active = true
      |<------------------- list of RouteResponse ------|        AND organization.status = ACTIVE
      |  build routing table, apply per-route rules     |
```

The endpoint and the account exist, and the UI's **Active routes** page shows exactly this feed. The gateway service does not call it yet. A route reaches the feed only when all three are true: the route is active, its organization is `ACTIVE`, and the route has not been deleted.

What the gateway is expected to do with each route (design intent, not implemented): match the incoming path against `pathPattern`, check `allowedMethods`, enforce `requestsPerMinute` and `idempotencyEnabled` (Redis), apply `headerRules`, forward to `targetBaseUrl + targetPathPrefix` with `responseTimeoutMs`, and publish usage events.

7.6 Suspending a tenant

Step	Effect
Platform admin sets the organization to `SUSPENDED`	Audit `CHANGE_ORGANIZATION_STATUS`
Gateway fetches routes	The tenant's routes are no longer returned
Tenant admin with an existing token calls any tenant endpoint	Refused: "The organization is not active"
Tenant admin tries to sign in	Refused: "The organization is not active"
Platform admin sets the organization back to `ACTIVE`	Everything works again, no data was removed

7.7 Deleting a tenant

`DELETE /api/platform/organizations/{id}` sets status `DELETED` and `deleted_at`. Nothing else is removed: routes, versions, audit records and the tenant administrator remain in the database. Because the organization is no longer `ACTIVE`, the tenant cannot sign in and the gateway stops receiving its routes. The organization disappears from the platform list and from `GET .../{id}`.

7.8 Session lifecycle in the UI

1. Login stores the session in `sessionStorage` and navigates to the role's home page.

2. A timer is set for the session expiry. A countdown is shown in the **Your session** card.
3. At expiry, or on any 401 from the API, the session is cleared and the login page shows "Your session has expired. Please sign in again."
4. **Sign out** clears the session immediately. Closing the tab also ends the session.

8. Known Issues and Roadmap

Items are marked **Verified** when confirmed by reading the code or schema, and **Inferred** when they follow from the code but were not executed. Severity is a suggestion.

8.1 Known issues

1. Business errors are not translated to HTTP statuses

Verified: there is no `@ControllerAdvice`, `@ExceptionHandler` or `ResponseStatusException` in the repository. **Inferred:** `ApiException` (401, 403, 404, 409, 400 for business rules) therefore surfaces as a generic 500 instead of the intended status and message. Effects: wrong credentials, "organization not active", duplicate route id and "route not found" are all reported as server errors; the UI falls back to its own per-call messages for some statuses but cannot tell these cases apart when it receives 500. Bean-validation failures do return 400. **Severity: high.** Fix: add a `@RestControllerAdvice` mapping `ApiException` to `ResponseEntity` with the exception's status and a `{message}` body.

2. Re-creating a deleted route fails

Verified: `route_configuration_versions` is unique on `(organization_id, route_id, version_number)` and survives route deletion by design; a new route with the same `routeId` starts at version 1 and writes a `CREATED` version 1 again. **Inferred:** the second create fails with a constraint violation. Workaround: use a different route id. Fix: derive the next version from the highest stored version for that route id.

3. Gateway and analytics services cannot validate tokens

Verified: their `SecurityConfig` has no `oauth2ResourceServer` and no `JwtDecoder` or `security.jwt.*` property exists. Any endpoint added there would reject valid tokens. **Severity: high** before adding features.

4. Seeded accounts and early secrets

Verified: the database password and JWT secret now come from `DB_PASSWORD` and `JWT_SECRET` (git-ignored `.env` or environment). The seeded `platform-admin` and `gateway-service` password hashes are still committed in `V3`. Change those passwords after first start, and use a fresh `JWT_SECRET` for any shared deployment because the earlier value was stored in plain text during development. **Severity: medium.**

5. Tokens cannot be revoked

No refresh, logout or revocation. A disabled user keeps a working token until it expires (tenant data access is still blocked if the organization stops being active). Lifetime is one hour.

6. Shared symmetric signing key

HS256 means any service that verifies tokens holds the key that can also forge them. Consider RS256/ES256 with a published public key once other services validate tokens.

7. Allowed-method validation is weak

Inferred: the service checks methods with Spring's `HttpMethod.valueOf`, which does not reject unknown names in current Spring versions, so a request can save a non-standard method. The UI only offers the seven standard methods.

8. Platform actions appear in the tenant's audit log

Verified: platform operations are recorded against the affected organization, so tenants see entries written by platform administrators (actor shown). This may be desirable (transparency) but is not documented as a decision.

9. Audit records only success

Verified: `AuditService.record` always writes `result = SUCCESS`, and only after the operation succeeds. Failures are not audited. The audit list is not paginated.

10. No pagination anywhere

Organizations, routes, versions, audit records and the gateway feed return full lists. Audit records and versions grow without bound.

11. Organization status is unrestricted

Any of `PENDING`, `ACTIVE`, `SUSPENDED` can be set from any other; there is no state machine or reason field.

12. Tenant administrator update requires a password

`PUT .../tenant-admin` needs a password every time; there is no partial update or dedicated password-change call.

13. One tenant administrator per organization

Enforced by a unique constraint. Tenant user management (the **Users** page) is a placeholder because the backend has no endpoints for it.

14. Inconsistent health responses and mislabelled config

The three `/health` responses use different field names, and `info.app.*` in `api-management-service` and `analytics-service` is copied from the gateway ("Custom API Gateway for routing, security and processing"). `HealthService` swallows database errors with `System.out.println` rather than a logger.

15. Tests

Each service has only the generated `contextLoads` test, which needs a reachable MySQL and the committed configuration. The UI has no test runner. There are no automated tests for route management, tenant isolation or the UI.

16. Repository hygiene

The workspace has no `.git` directory, no CI configuration, no Docker or compose file, and no root build file. The UI components folder is spelled `componenets`.

8.2 Roadmap suggestions

Ordered by dependency, not by effort.

1. **Fix error handling** (issue 1) and add controller tests for the tenant endpoints. This unblocks meaningful UI error messages.
2. **Fix route re-creation** (issue 2) and add a test for create, delete, create.
3. **Externalize secrets** (issue 4).
4. **Implement the data plane.** Gateway logs in as `gateway-service`, polls `GET /api/gateway/routes`, builds a routing table, proxies with `RestClient`, applies header rules, rate limits and idempotency with Redis. Requires JWT validation (issue 3).
5. **Add events and analytics.** Define Kafka topics for request outcomes from the gateway, consume them in `analytics-service`, store in MongoDB, expose query endpoints and add an analytics page to the UI.
6. **Tenant user management and gateway registration** endpoints, then replace the two UI placeholders.
7. **Operational quality:** pagination, audit of failures, token revocation or shorter lifetimes with refresh, CI pipeline, container images, UI tests.

8.3 Documentation maintenance

When the code changes, update the matching file: endpoints in `03`, tables in `04`, screens in `06`, scenarios in `07`, and the status table in `README.md`. Rebuild the PDF afterwards (`docs/tools`, see 02 - Getting started). `docs/index.html` is edited by hand.